JAVA LAB RECORD

Java Collection Framework: Important Methods by Interface

Aim • Description • UML Diagram • Commented Source Code • Terminal Output • Result

Program file name: CollectionInterfacesDemo.java

Class name: CollectionInterfacesDemo

Department / Subject: _____

Name: _____ Roll No.: _____

1. Aim

To demonstrate important methods of the Java Collection Framework interfaces—Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator and ListIterator—using simple Java examples.

2. Description

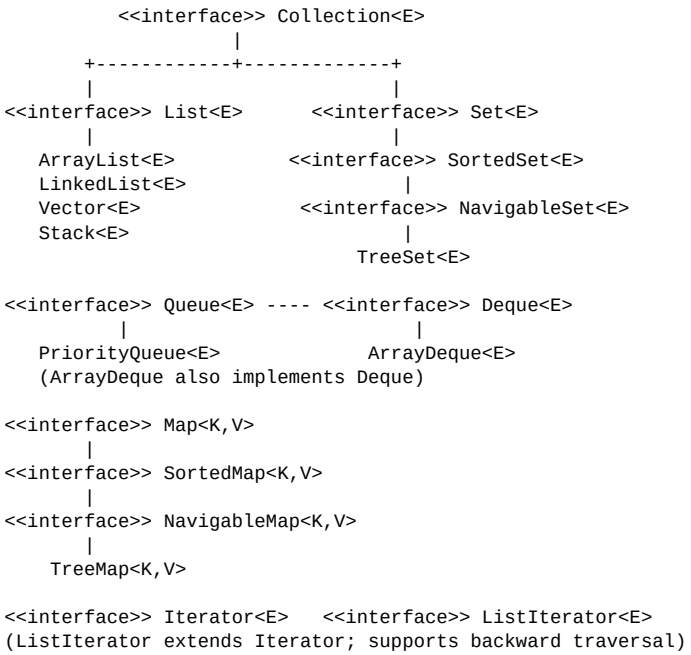
The Java Collections Framework provides interfaces and classes to store, retrieve and process groups of objects. List preserves sequence and allows duplicates; Set stores unique elements; SortedSet/NavigableSet provide sorted and navigation operations; Queue processes elements at its head; Deque supports both ends; Map stores key-value pairs; SortedMap/NavigableMap order keys; Iterator and ListIterator traverse collections.

The program uses ArrayList, TreeSet, ArrayDeque and TreeMap because they demonstrate most of these interface methods clearly. Methods such as add, remove, contains, size and clear are demonstrated with collections. Iterator and ListIterator examples show forward/backward traversal and safe modification. Views returned by headSet, tailSet, subSet, headMap, tailMap and subMap are backed by their original collection/map.

Important notes

• Map is a separate interface; it does not extend Collection.
• TreeSet implements NavigableSet and TreeMap implements NavigableMap.
• Natural ordering is used here, so comparator() returns null for TreeSet and TreeMap.
• Queue is commonly FIFO, but exact ordering depends on the implementation (for example, PriorityQueue orders by priority).
• Avoid modifying a collection directly while iterating over it; use Iterator.remove() when removal during iteration is needed.

3. UML Diagram (simplified)



4. Source Code with Comments

Save the following program as **CollectionInterfacesDemo.java**. The public class name and file name must match.

```
import java.util.*;

public class CollectionInterfacesDemo {
    public static void main(String[] args) {
        // COLLECTION and LIST
        List<String> list = new ArrayList<>();
        list.add("Mango");                 // add(E)
        list.add("Apple");
        list.add("Mango");                 // List allows duplicates
        list.add(1, "Banana");             // add(index, element)
        System.out.println("List: " + list);
        System.out.println("get(1): " + list.get(1));
        list.set(1, "Orange");             // replace at index
        System.out.println("set(1, Orange): " + list);
        System.out.println("indexOf(Mango): " + list.indexOf("Mango"));
        System.out.println("lastIndexOf(Mango): " + list.lastIndexOf("Mango"));
        System.out.println("subList(0, 2): " + list.subList(0, 2));
        list.sort(Comparator.naturalOrder()); // sort ascending
        System.out.println("Sorted list: " + list);

        Collection<String> extra = new ArrayList<>();
        extra.add("Grapes");
        extra.add("Apple");
        list.addAll(extra);                // addAll(Collection)
        System.out.println("contains(Apple): " + list.contains("Apple"));
        System.out.println("containsAll(extra): " + list.containsAll(extra));
        System.out.println("size: " + list.size());
        System.out.println("isEmpty: " + list.isEmpty());
        Iterator<String> it = list.iterator();
        System.out.print("Iterator: ");
        while (it.hasNext()) System.out.print(it.next() + " ");
        System.out.println();
        list.remove("Grapes");             // remove(Object)
        list.removeAll(extra);             // remove all matching elements
        System.out.println("After remove/removeAll: " + list);

        // SET, SORTEDSET, NAVIGABLESET (TreeSet implements these)
        NavigableSet<Integer> numbers = new TreeSet<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        numbers.add(20);                   // duplicate ignored
        System.out.println("\nTreeSet: " + numbers);
        System.out.println("first/last: " + numbers.first() + "/" + numbers.last());
        System.out.println("headSet(30): " + numbers.headSet(30));
        System.out.println("tailSet(30): " + numbers.tailSet(30));
        System.out.println("subSet(20, 40): " + numbers.subSet(20, 40));
        System.out.println("comparator (natural order): " + numbers.comparator());
        System.out.println("lower/floor/ceiling/higher(30): "
                + numbers.lower(30) + "/" + numbers.floor(30) + "/"
                + numbers.ceiling(30) + "/" + numbers.higher(30));
        System.out.println("descendingSet: " + numbers.descendingSet());
        System.out.println("pollFirst: " + numbers.pollFirst());
        System.out.println("pollLast: " + numbers.pollLast());
        System.out.println("Set contains 20: " + numbers.contains(20));
        System.out.println("Set size: " + numbers.size());
        numbers.clear();
        System.out.println("Set empty after clear: " + numbers.isEmpty());

        // QUEUE and DEQUE
        Queue<String> queue = new ArrayDeque<>();
        queue.add("A");
        queue.offer("B");
        queue.offer("C");
        System.out.println("\nQueue: " + queue);
        System.out.println("element/peek: " + queue.element() + "/" + queue.peek());
        System.out.println("remove/poll: " + queue.remove() + "/" + queue.poll());
        Deque<String> deque = new ArrayDeque<>();
        deque.addFirst("Middle");
        deque.addFirst("Front");
        deque.addLast("Back");
        deque.offerFirst("NewFront");
        deque.offerLast("NewBack");
        System.out.println("Deque: " + deque);
        System.out.println("peekFirst/peekLast: " + deque.peekFirst() + "/" + deque.peekLast());
        System.out.println("pollFirst/pollLast: " + deque.pollFirst() + "/" + deque.pollLast());
```

```
        System.out.println("Deque after polls: " + deque);

        // MAP, SORTEDMAP, NAVIGABLEMAP (TreeMap implements these)
        NavigableMap<Integer, String> map = new TreeMap<>();
        map.put(3, "Three");
        map.put(1, "One");
        map.put(2, "Two");
        map.put(4, "Four");
        System.out.println("\nTreeMap: " + map);
        System.out.println("get(2): " + map.get(2));
        System.out.println("containsKey(3): " + map.containsKey(3));
        System.out.println("containsValue(Two): " + map.containsValue("Two"));
        System.out.println("keySet: " + map.keySet());
        System.out.println("values: " + map.values());
        System.out.println("entrySet: " + map.entrySet());
        System.out.println("size/isEmpty: " + map.size() + "/" + map.isEmpty());
        System.out.println("firstKey/lastKey: " + map.firstKey() + "/" + map.lastKey());
        System.out.println("headMap(3): " + map.headMap(3));
        System.out.println("tailMap(2): " + map.tailMap(2));
        System.out.println("subMap(2, 4): " + map.subMap(2, 4));
        System.out.println("comparator: " + map.comparator());
        System.out.println("lower/floor/ceiling/higherKey(2): "
                + map.lowerKey(2) + "/" + map.floorKey(2) + "/"
                + map.ceilingKey(2) + "/" + map.higherKey(2));
        System.out.println("firstEntry/lastEntry: " + map.firstEntry() + "/" + map.lastEntry());
        System.out.println("descendingMap: " + map.descendingMap());
        System.out.println("pollFirstEntry: " + map.pollFirstEntry());
        System.out.println("pollLastEntry: " + map.pollLastEntry());
        map.remove(3);
        System.out.println("After remove(3): " + map);
        map.clear();
        System.out.println("Map empty after clear: " + map.isEmpty());

        // LISTITERATOR supports forward and backward movement
        List<String> words = new ArrayList<>(Arrays.asList("One", "Two", "Three"));
        ListIterator<String> li = words.listIterator();
        System.out.print("\nListIterator forward: ");
        while (li.hasNext()) System.out.print(li.next() + " ");
        System.out.print("\nListIterator backward: ");
        while (li.hasPrevious()) System.out.print(li.previous() + " ");
        System.out.println();
        li = words.listIterator();
        li.next();
        li.add("Added");                    // insert at iterator position
        li.next();
        li.set("Changed");                  // replace last returned element
        System.out.println("After ListIterator add/set: " + words);
        System.out.println("nextIndex/previousIndex: " + li.nextIndex() + "/" + li.previousIndex());
        li = words.listIterator();
        li.next();
        li.remove();                        // remove last returned element
        System.out.println("After ListIterator remove: " + words);

        // Iterator forEachRemaining and remove
        Iterator<String> iterator = words.iterator();
        System.out.print("forEachRemaining: ");
        iterator.forEachRemaining(s -> System.out.print(s + " "));
        System.out.println();

        // Clear and demonstrate Collection methods
        Collection<String> sample = new ArrayList<>(Arrays.asList("X", "Y", "Z"));
        System.out.println("sample contains Y: " + sample.contains("Y"));
        sample.remove("Y");
        System.out.println("sample after remove: " + sample);
        sample.clear();
        System.out.println("sample after clear: " + sample);
    }
}
```

5. Compile and Run

```
javac CollectionInterfacesDemo.java
java CollectionInterfacesDemo
```

6. Terminal Output Screenshot

The following terminal-style screenshots show the program output. Output order is deterministic for the ArrayList, TreeSet, ArrayDeque and TreeMap examples used here.

```
Terminal — CollectionInterfacesDemo

$ java CollectionInterfacesDemo
List: [Mango, Banana, Apple, Mango]
get(1): Banana
set(1, Orange): [Mango, Orange, Apple, Mango]
indexOf(Mango): 0
lastIndexOf(Mango): 3
subList(0, 2): [Mango, Orange]
Sorted list: [Apple, Mango, Mango, Orange]
contains(Apple): true
containsAll(extra): true
size: 6
isEmpty: false
Iterator: Apple Mango Mango Orange Grapes Apple
After remove/removeAll: [Mango, Mango, Orange]

TreeSet: [10, 20, 30, 40]
first/last: 10/40
headSet(30): [10, 20]
tailSet(30): [30, 40]
subSet(20, 40): [20, 30]
comparator (natural order): null
lower/floor/ceiling/higher(30): 20/30/30/40
descendingSet: [40, 30, 20, 10]
pollFirst: 10
pollLast: 40
Set contains 20: true
Set size: 2
Set empty after clear: true

Queue: [A, B, C]
element/peek: A/A
remove/poll: A/B
Deque: [NewFront, Front, Middle, Back, NewBack]
peekFirst/peekLast: NewFront/NewBack
```

Terminal — CollectionInterfacesDemo

```
$ java CollectionInterfacesDemo
pollFirst/pollLast: NewFront/NewBack
Deque after polls: [Front, Middle, Back]

TreeMap: {1=One, 2=Two, 3=Three, 4=Four}
get(2): Two
containsKey(3): true
containsValue(Two): true
keySet: [1, 2, 3, 4]
values: [One, Two, Three, Four]
entrySet: [1=One, 2=Two, 3=Three, 4=Four]
size/isEmpty: 4/false
firstKey/lastKey: 1/4
headMap(3): {1=One, 2=Two}
tailMap(2): {2=Two, 3=Three, 4=Four}
subMap(2, 4): {2=Two, 3=Three}
comparator: null
lower/floor/ceiling/higherKey(2): 1/2/2/3
firstEntry/lastEntry: 1=One/4=Four
descendingMap: {4=Four, 3=Three, 2=Two, 1=One}
pollFirstEntry: 1=One
pollLastEntry: 4=Four
After remove(3): {2=Two}
Map empty after clear: true

ListIterator forward: One Two Three
ListIterator backward: Three Two One
After ListIterator add/set: [One, Added, Changed, Three]
nextIndex/previousIndex: 3/2
After ListIterator remove: [Added, Changed, Three]
forEachRemaining: Added Changed Three
sample contains Y: true
sample after remove: [X, Z]
sample after clear: []
```

7. Result

Thus, the important methods of the Java Collection Framework interfaces were demonstrated successfully using Java program examples. The program illustrates collection operations, list indexing and sorting, set uniqueness and navigation, queue/deque operations, map key-value operations, and iterator-based traversal and modification.

Conclusion: These interfaces and methods help programmers choose the appropriate data structure for storing and processing data efficiently.